Graphs and Graph Traversals

[CSE2010] Data Structures

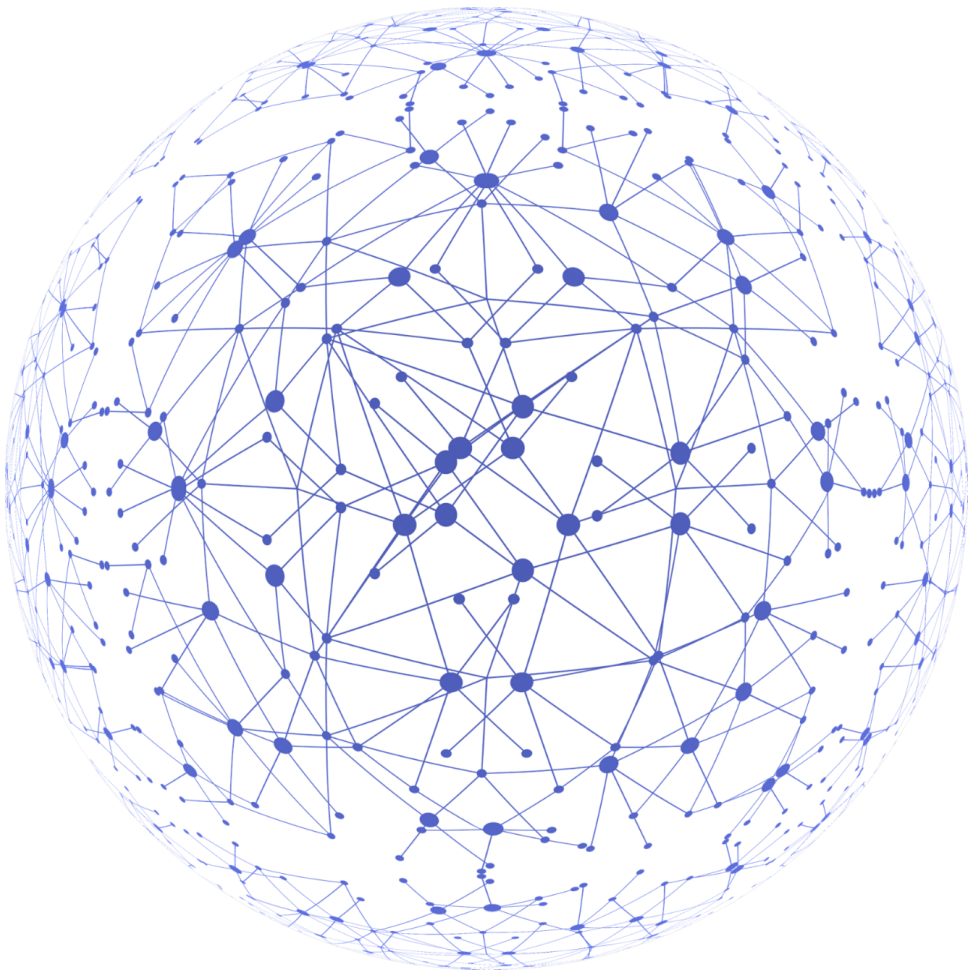
Department of Data Science

Graph

A way of representing relationships that exist between pairs of objects

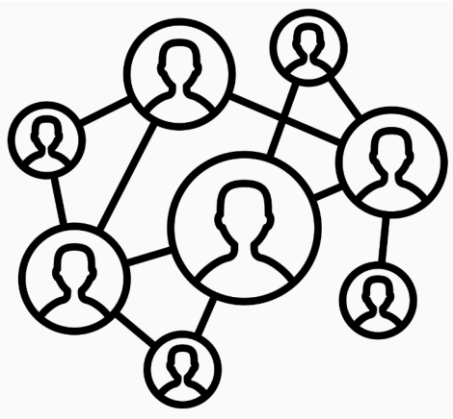
Graphs

- Graphs are a general language to describe and analyze entities with relations/interactions



Real-World Graph Data

- Many types of data are represented as **graphs**.
- Complex domains have a rich relational structure, which can be represented as a relational graph.



Social Networks

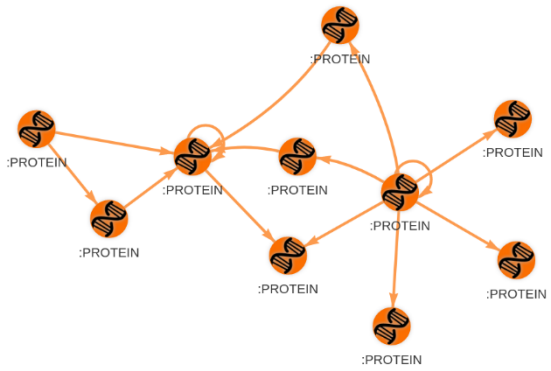
Image credit: thenounproject

Traffic, Subway Maps

Image credit: medium

Computer Networks

Image credit: salientnetworks

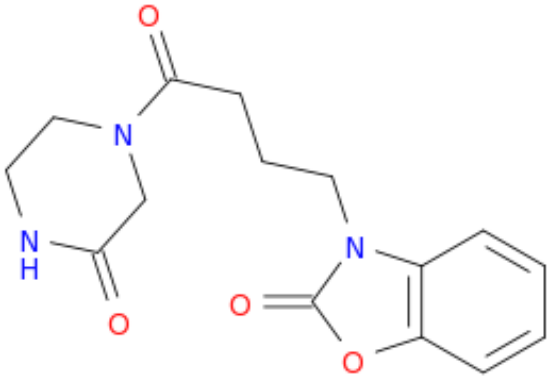


Protein-Protein Interaction

Image credit: memgraph

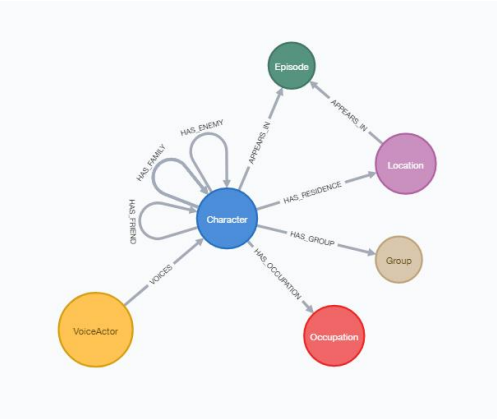
Real-World Graph Data

- Many types of data are represented as **graphs**.
- Complex domains have a rich relational structure, which can be represented as a relational graph.



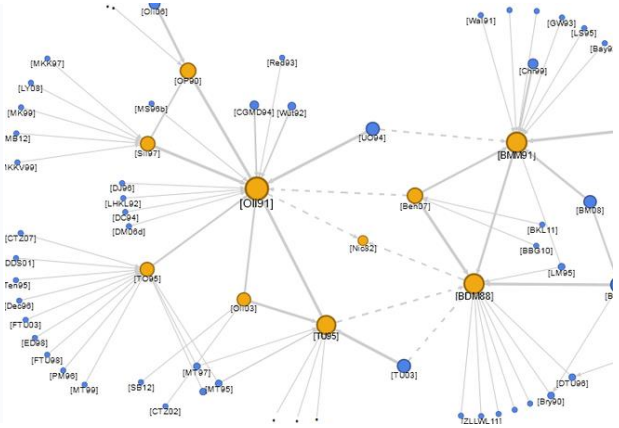
Chemical Compounds

Image credit: molinstincts



Knowledge Graphs

Image credit: nielsdejong



Citation Networks

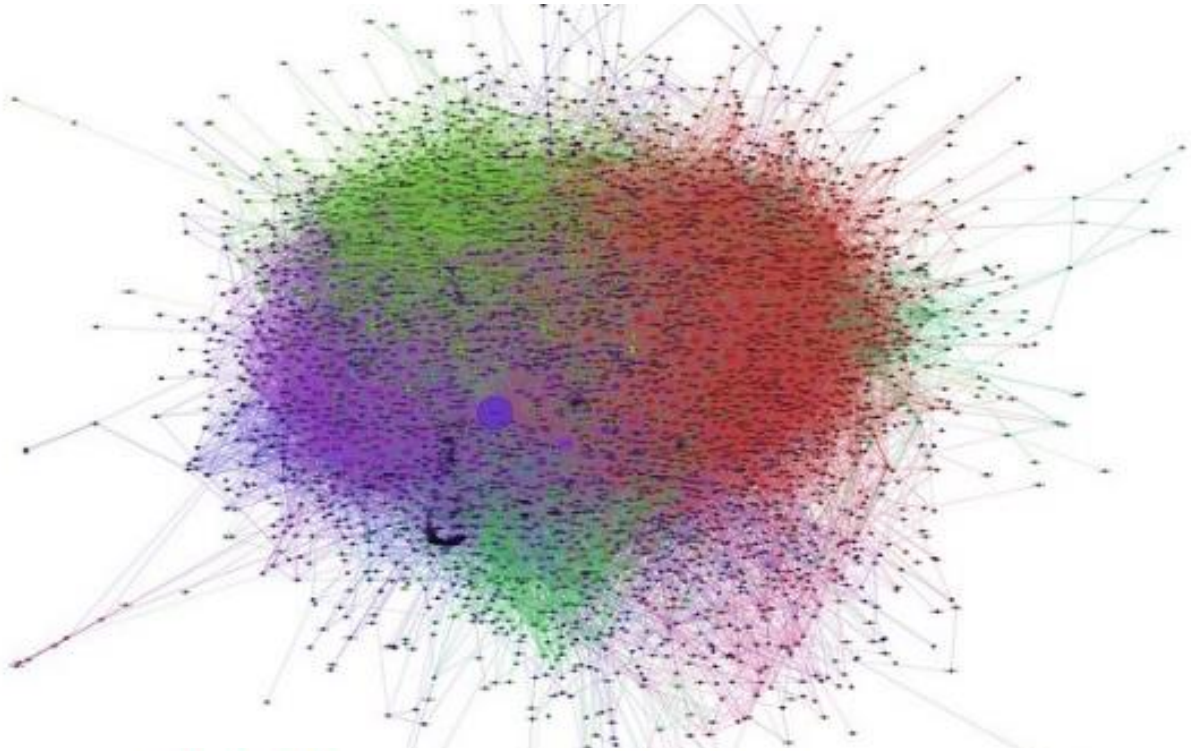
Image credit: researchgate

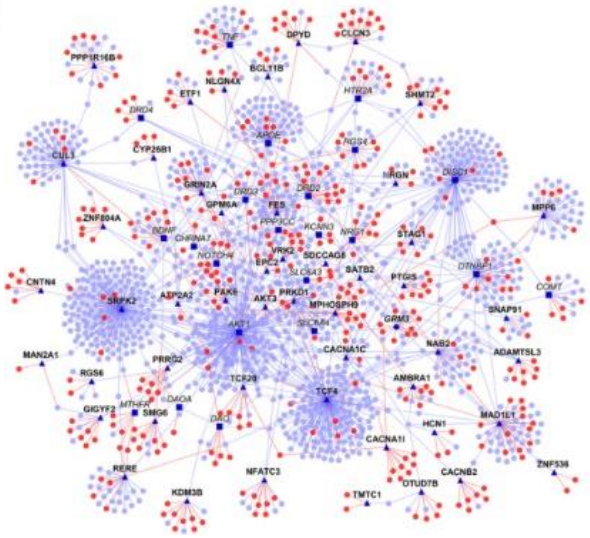
Networks of Neurons

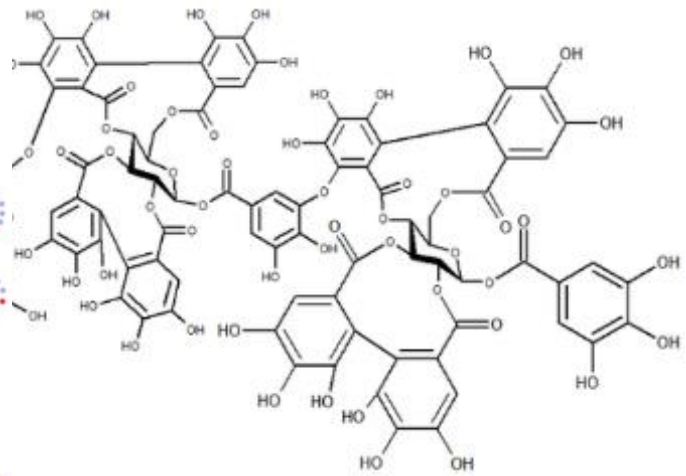
Image credit: medium

What Do We Find from Graph Data?

- Social Networks
  - Could we locate the communities where each person in the community know each other?
    - Connected components
    - Triangle count and clustering coefficient.
  - What is the minimum number of hops to reach the person?
    - The shortest path problem
- Protein-protein interaction networks or chemical compounds
  - How can we find the pattern (frequently) occurred in the graphs?
    - Subgraph matching
  - Can we reach out the target node from current node?
    - Reachability
  - What is the minimum number of hops to reach the target node from the current node?
    - The shortest path problem

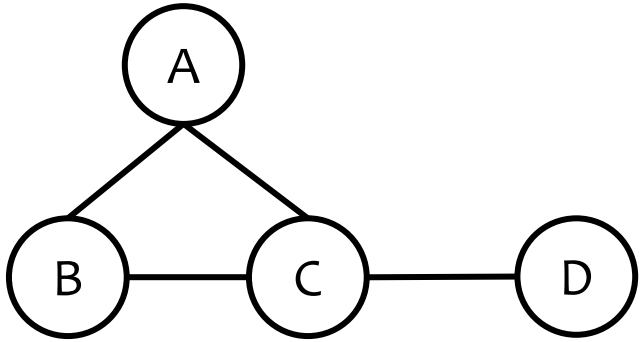






Graphs

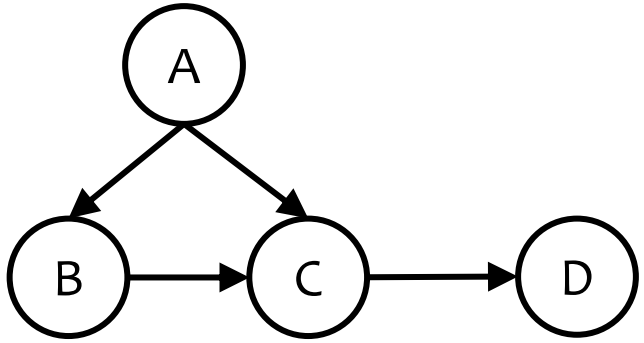
- Graph is a collection V of vertices (or nodes), and a collection E of edges each of which connects a pair of vertices.
  - An edge is represented as (u, v) where $u, v \in V$



- $G = (V, E)$
  - $V = \{A, B, C, D\}$
  - $E = \{(A, B), (A, C), (B, C), (C, D)\}$

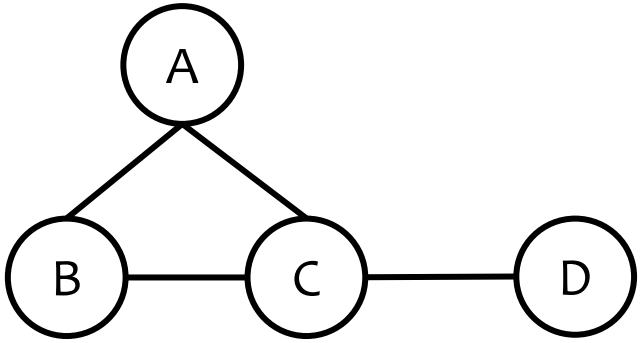
Edges are either Directed or Undirected

- Edge (u, v) is directed from u to v if u precedes v; undirected otherwise.



Directed graph (digraph):

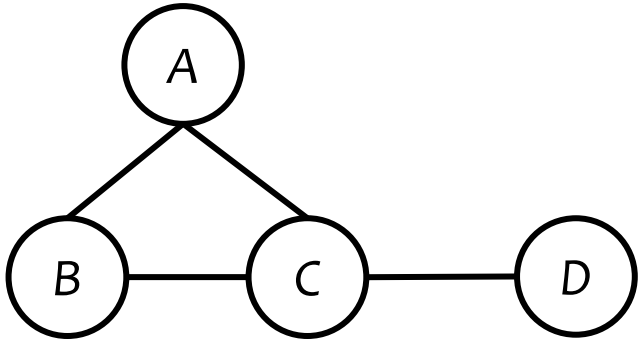
a graph with directed edges



Undirected graph:

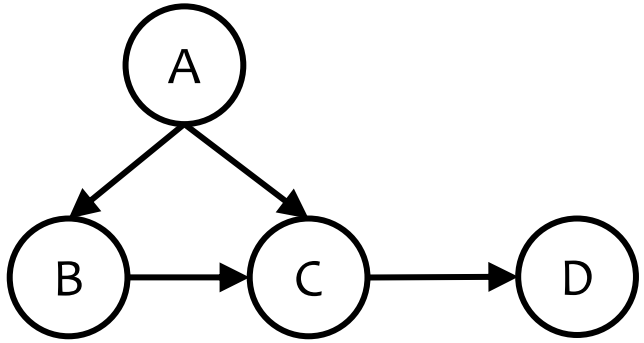
a graph with undirected edges

Graph Terminologies



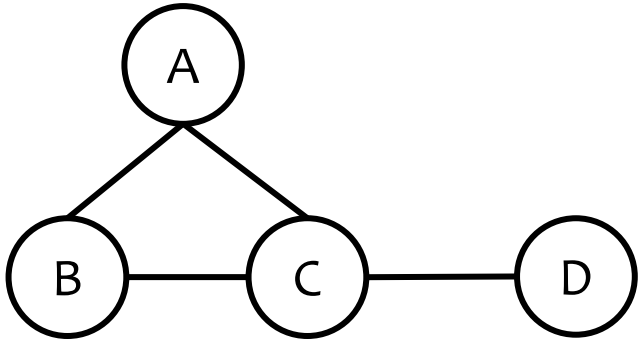
- Two vertices joined by an edge are called the **endpoints**.
  - For $(u, v) \in E$, u and v are endpoints.
- Two vertices u and v are **adjacent** if there is an edge whose endpoints are u and v.
  - A and B are adjacent.
- An edge is **incident** to a vertex if the vertex is one of the edge's endpoint.
  - (A, B) is incident to A and B.
- The **degree** of a vertex v, $\deg(v)$, is the number of incident edges of v.
  - $\deg(C) = 3$

Directed Graphs



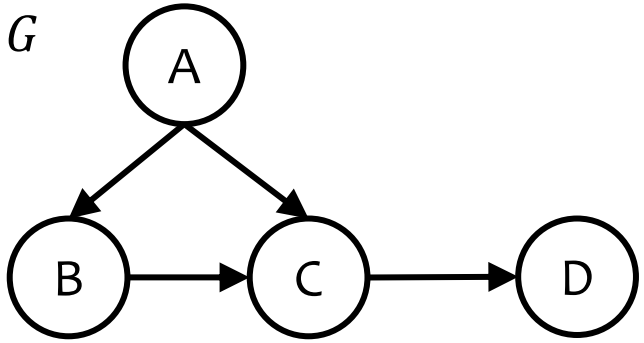
- For directed edge (u, v), the first endpoint u is the **origin**, and the other v is the **destination** of the edge.
  - A is the origin, B is the destination of edge (A, B).
- The **outgoing edges** of a vertex are the directed edges whose origin is that vertex.
  - The outgoing edges of A is (A,B) and (A,C).
- The **incoming edges** of a vertex are the directed edges whose destination is that vertex.
  - The incoming edge of D is (C,D).
- The **in-degree** and **out-degree** of a vertex v are the number of the incoming and outgoing edges of v, respectively.

Connectivity of Graphs



- Given a graph $G = (V, E)$
- **Path** is a sequence of adjacent vertices.
  - A-B-C-D
- The **length** of path is the number of edges in the path.
- **Cycle** is a path that starts and ends at the same vertex, and that includes at least one edge.
  - A-B-C-A
- Path is **simple** if each vertex in the path is distinct.

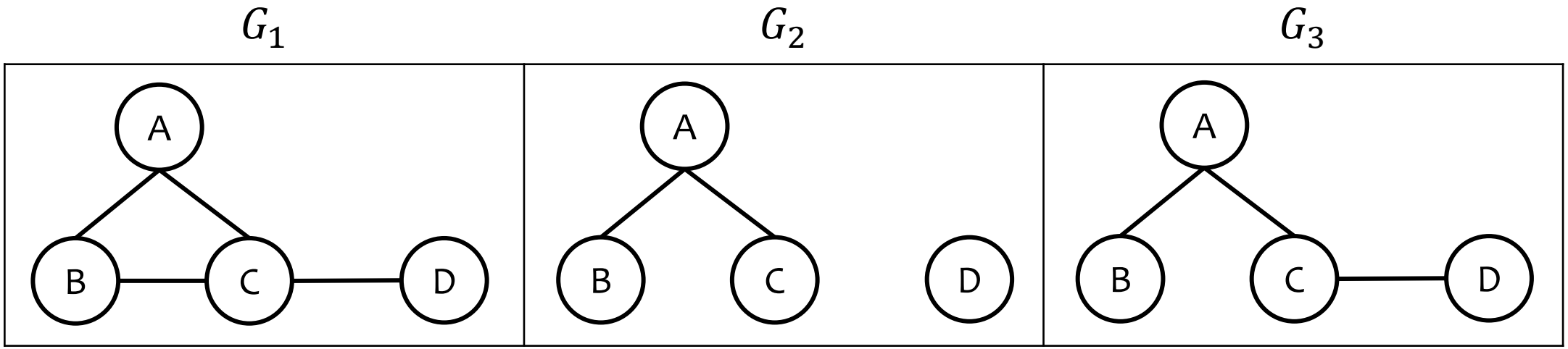
Connectivity of Graphs



- Given a directed graph $G = (V, E)$
- **Directed path** is a path such that all edges are directed and are traversed along their direction.
  - A-B-C-D
- Directed acyclic graph (**DAG**) is a directed graph with no directed cycles.
  - G is a DAG

Graphs vs. Trees

- While all trees are graphs, not all graphs are trees.
- For a graph to be considered a tree, it cannot have cycles, and all nodes must be connected.



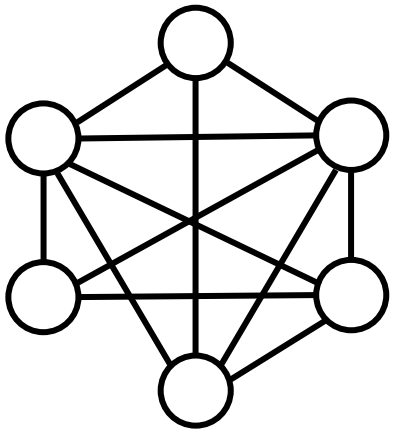
Complexity Analysis in Graphs

- Graph algorithm runtimes or spaces depend on $|V|$ and $|E|$.

$$O(|V| + |E|)$$
$$O(|V||E|)$$
$$O(|V|log|V|)$$
$$O\left(|V|^{3/2}\right)$$

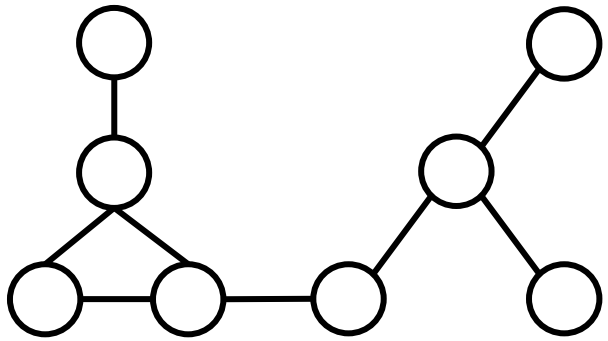
- Between $O\left(|V|^{3/2}\right)$ and $O(|E|)$, which one is faster?
  It depends on the density of a graph.

Density of Graphs



Dense graphs: $|E| \approx |V|^2$

A large fraction of pairs of vertices are connected by edges



Sparse graph: $|E| \approx |V|$

The number of edges is small relative to the number of vertices.

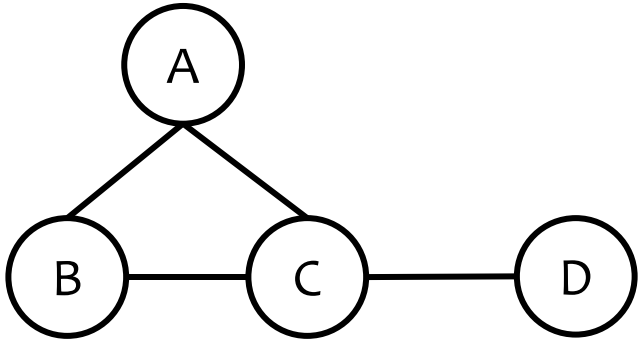
How Do We Represent Graphs?

Adjacency matrices

Adjacency lists

Adjacency Matrices

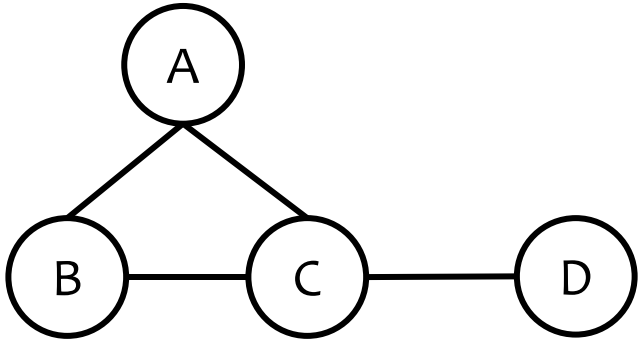
- $\boldsymbol{A}[\boldsymbol{u}][\boldsymbol{v}] = \mathbf{1}$ if there is an edge between u and v, 0 otherwise.
  - A table of size |V| (to store a mapping from vertex names to array indices) is needed.

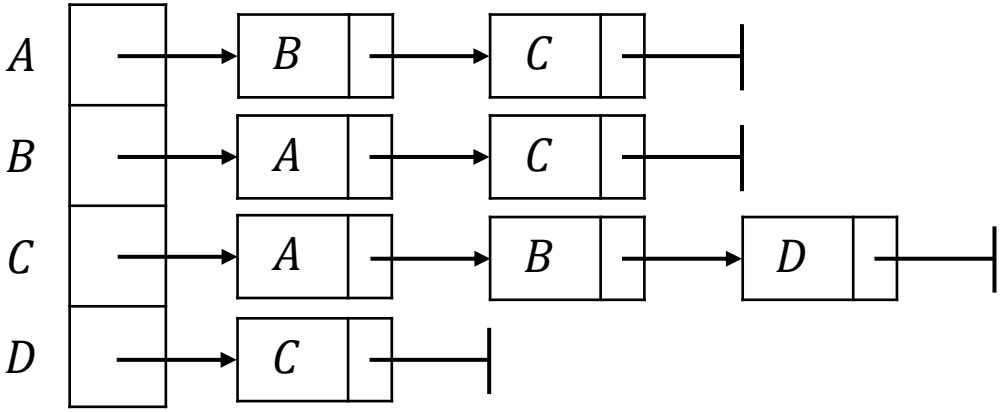


$\boldsymbol{A}$	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

Adjacency Lists

- For each vertex, keep a list of adjacent vertices





Which Representation is Better?

- Time complexity:
  Different operations are faster in different representations.
- Space complexity:
  Adjacency list is better.

	Time Complexity		
	Is Edge?	List Neighbors	List All Edges
Adj. Matrix	$O(1)$	$O(\|V\|)$	$O(\|V\|^2)$
Adj. List	$O(deg)$	$O(deg)$	$O(\|E\|)$

	Space Complexity
Adj. Matrix	$O(\|V\|^2)$
Adj. List	$O(\|V\| + \|E\|)$

Adjacency Matrices vs. Adjacency Lists

- Appropriate for
  - Dense graphs with $|E|$ close to $|V|^2$
  - Are two vertices adjacent?

- Appropriate for
  - Sparse graphs
  - Iterating neighbors of a vertex

- For many problems, consider adjacency lists.

Graph Traversals

A systematic procedure to explore a graph by examining all of its vertices and edges

Graph Traversals

- Key to answering many fundamental questions on graphs
  - How can we search for a particular vertex in a graph?
  - Are two vertices are connected in a graph?
  - What is a path from vertex u to vertex v, or report that no such path exists
  - Given a start vertex s of G, compute a path with the minimum number of edges between s and v for every vertex v of G
  - Whether G is connected
  - Compute a cycle in G, or report that G has no cycles
- Two methods
  - Depth-First Search
  - Breadth-First Search

Depth-First Search (DFS)

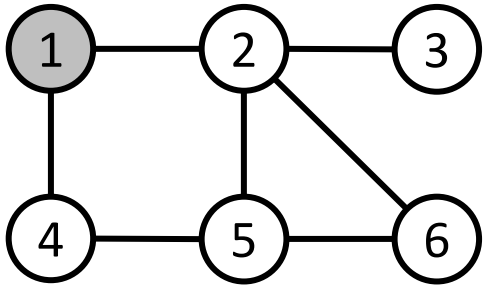
- Given a graph $G = (V, E)$ and a start vertex (or source vertex) s
- DFS searches deeper in the graph whenever possible.
  1. Explore edges out of the most recently discovered vertex v that still has unexplored edges leaving it.
  2. Once all of v's edges have been explored, backtrack to explore edges leaving the vertex from which v was discovered.
  3. Continue until we have discovered all the vertices that are reachable from the original source vertex.

DFS: Recursive & Iterative

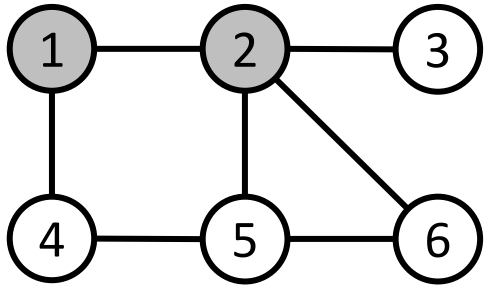
DFS-Recursive(G, u)	**DFS-Iterative (G, s)**

Mark u as visited
for each v adjacent to u
  if v is not visited
  DFS-Recursive(G, v)
	Build an empty stack S
	Push s into S
	while S is not empty
	  $u \leftarrow$ Pop element from S
	  if u is not visited
	  Mark u as visited
	  for each $v \in V$ adjacent to u
	  if v is not visited
	  Push v into S

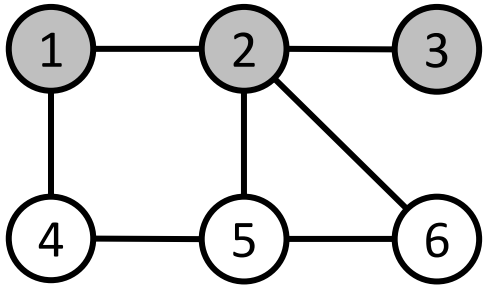
DFS-Recursive



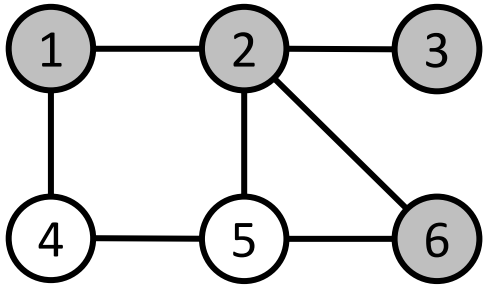
Visit the starting vertex 1.



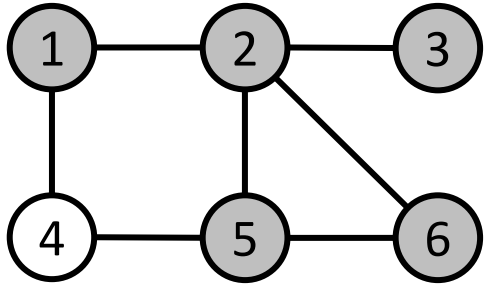
Visit the first neighbor 2 of 1.



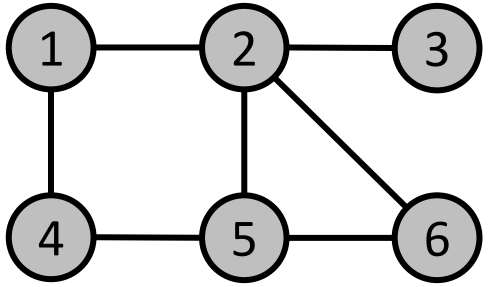
Visit the first neighbor 3 of 2.



Backtrack from 3. Visit the second neighbor 6 of 2.

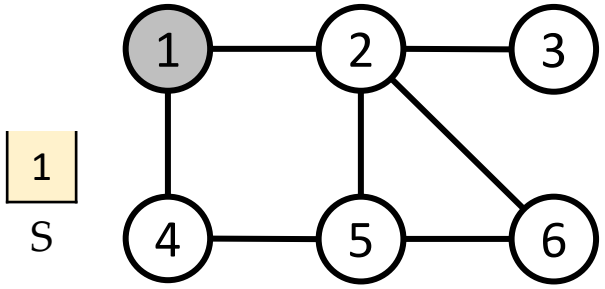


Visit the second neighbor 5 of 6 (as 2 is already visited).

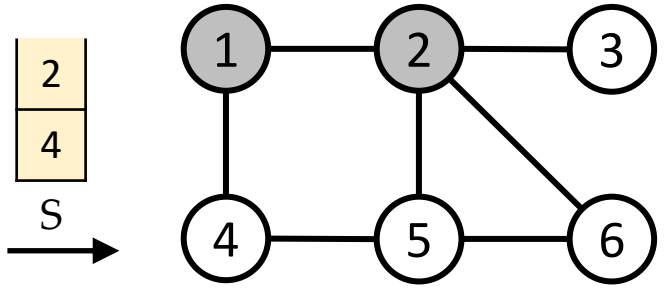


Visit the neighbor 4 of 5.

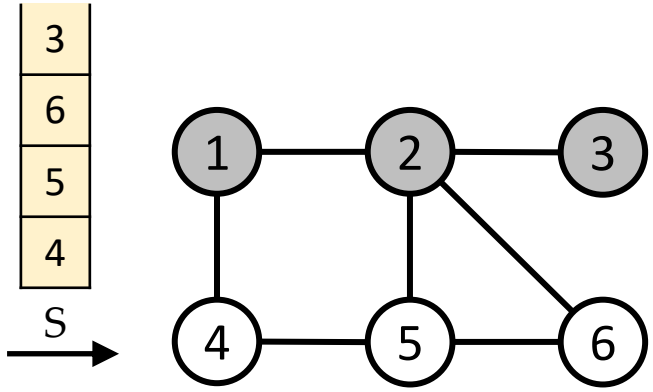
DFS-Iterative



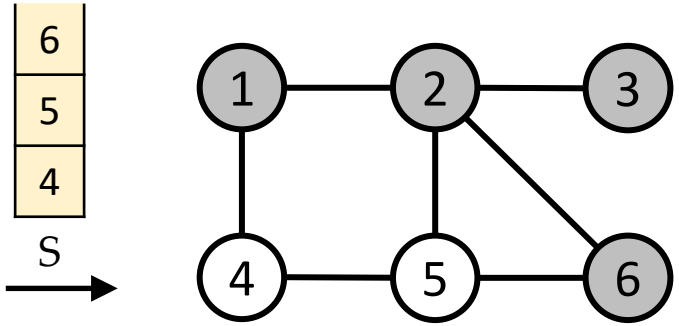
Visit the starting vertex 1.



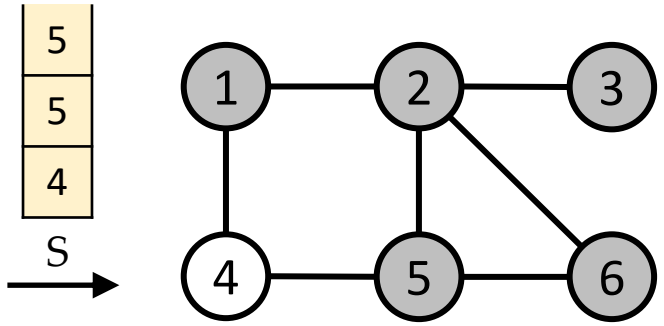
Visit the first neighbor 2 of 1.



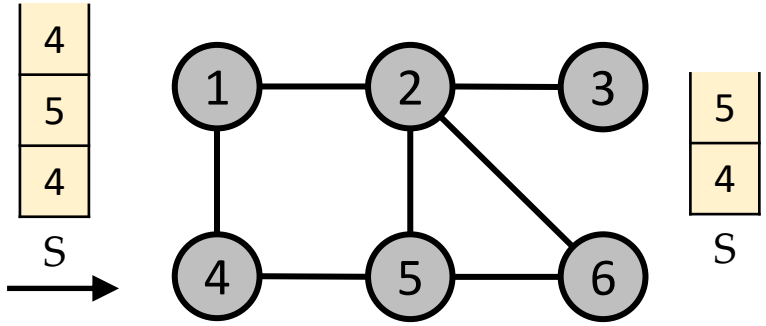
Visit the first neighbor 3 of 2.



Backtrack from 3. Visit the second neighbor 6 of 2.



Visit the second neighbor 5 of 6 (as 2 is already visited).



Visit the neighbor 4 of 5.

DFS

- Graphs can be disconnected.

DFS-Disconnected(G)

Mark u as unvisited for every $u \in V$

for each vertex $u \in V$

  if u is unvisited

    DFS-Recursive(G, u) // or DFS-Iterative(G, u)

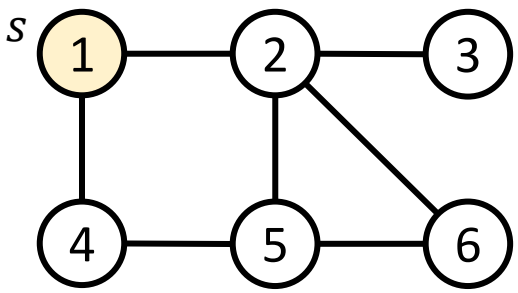
Breadth-First Search (BFS)

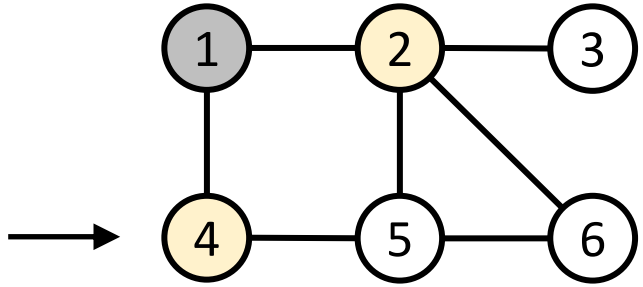
- Given a graph $G = (V, E)$ and a start vertex (or source vertex) s
- BFS explores the distance (smallest number of edges) from s to each reachable vertex.
  - Expand the frontier between discovered and undiscovered vertices uniformly across the breadth of the frontier.
  - Discover all vertices at distance k from s before discovering any vertices at distance k+1.

BFS

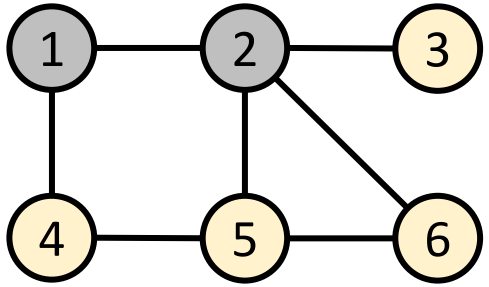
```
BFS(G, s)
  Build an empty queue Q
  Mark u as unvisited for every u ∈ V
  Mark source s as visited
  Insert s into Q
  while Q is not empty
    u ← Delete an element from Q
    for each v ∈ V adjacent to u
      if v is unvisited
        Mark v as visited
        Insert v into Q
```

BFS

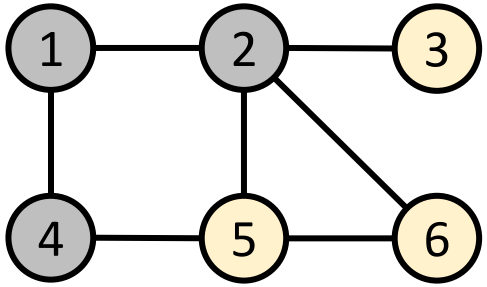




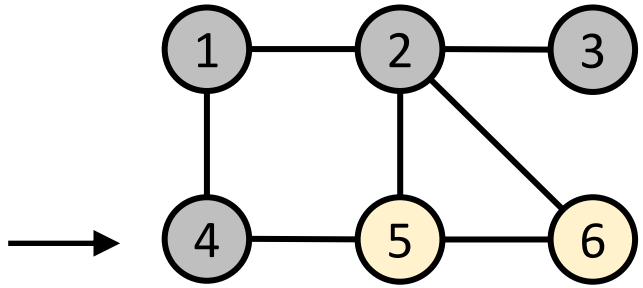
Q | 1



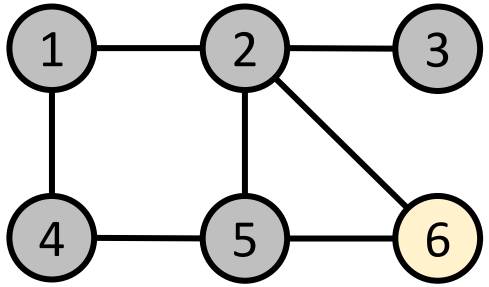
Q | 2 | 4



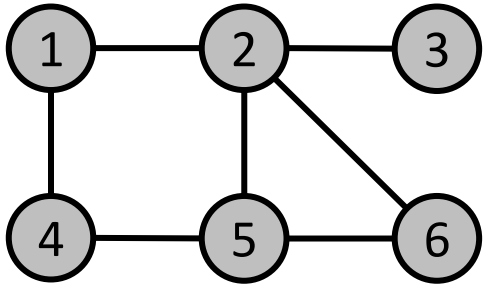
Q | 4 | 3 | 5 | 6



Q | 3 | 5 | 6



Q | 5 | 6



Q | 6

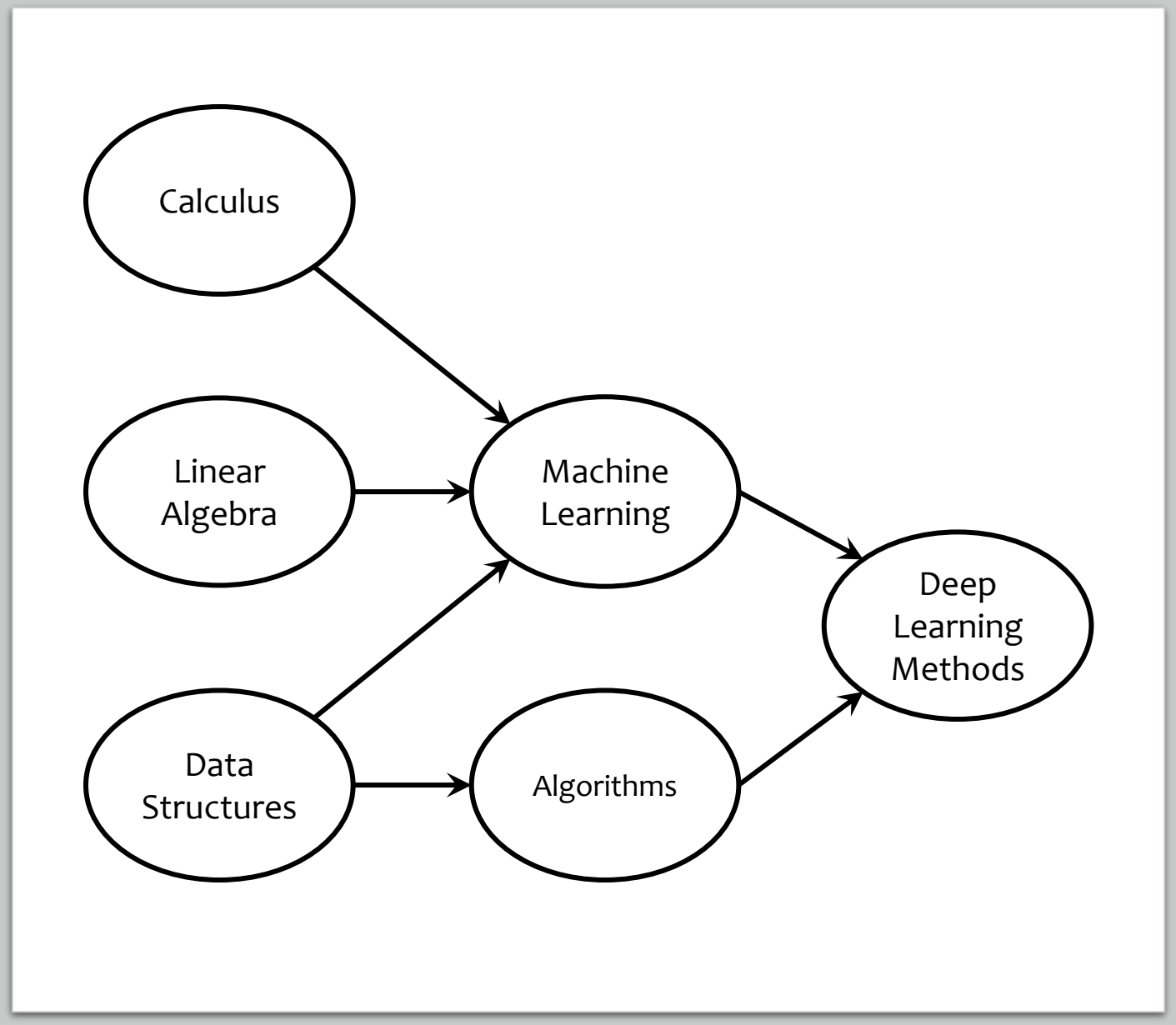
Q

Topological Sort

An ordering of vertices in a directed acyclic graph (DAG) in which each node comes before all nodes to which it has outgoing edges.

Directed Acyclic Graphs (DAGs)

- Directed graphs without directed cycles
- Applications
  - Prerequisites between courses
  - Inheritance between classes of an object-oriented program
  - Scheduling constraints between the tasks of a project



Topological Sort

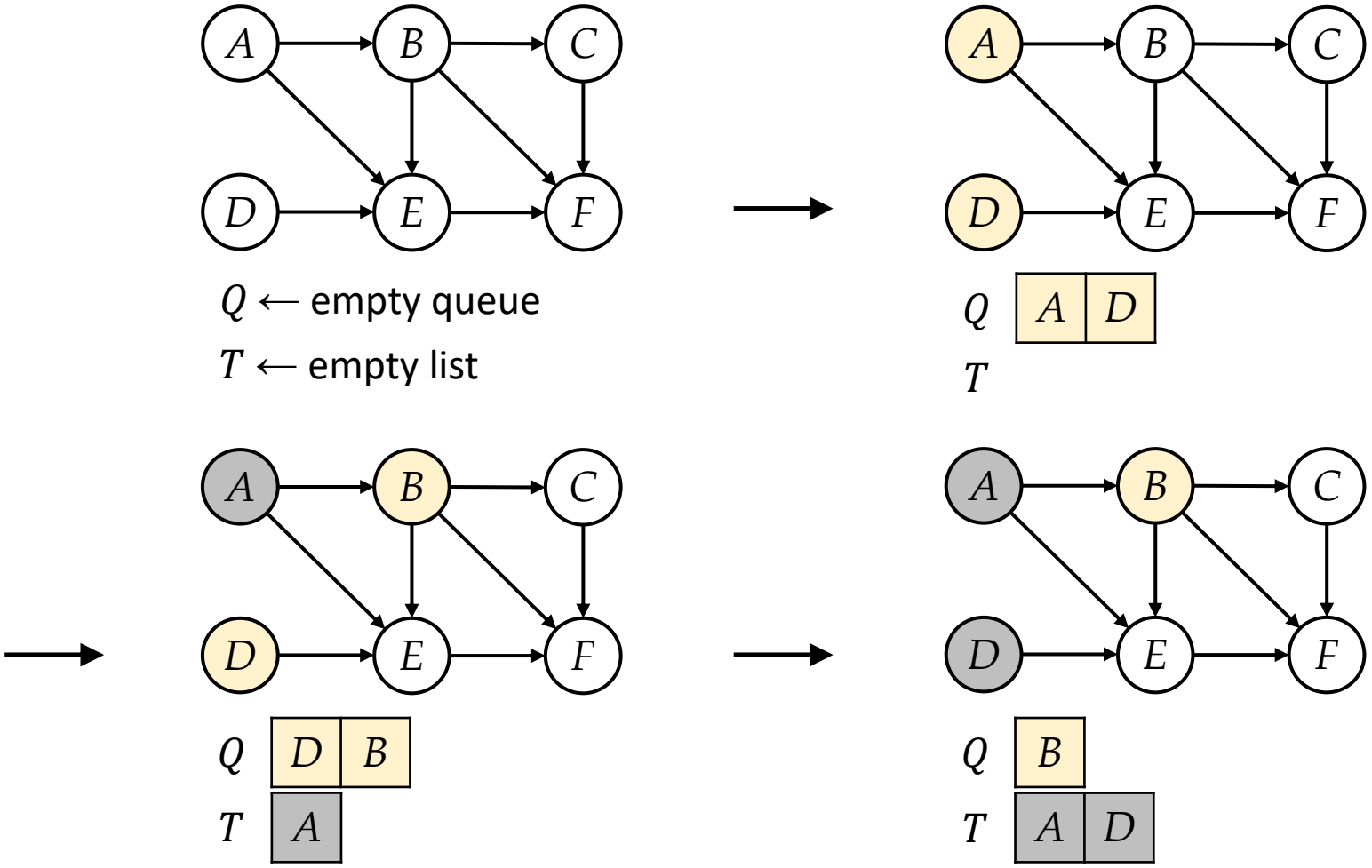
- Linear ordering of all the vertices in V such that if G contains an edge (u, v), then u appears before v in the ordering.
- Every DAG may have one or more topological orderings.
- Not possible if the graph has a cycle

Topological Sort

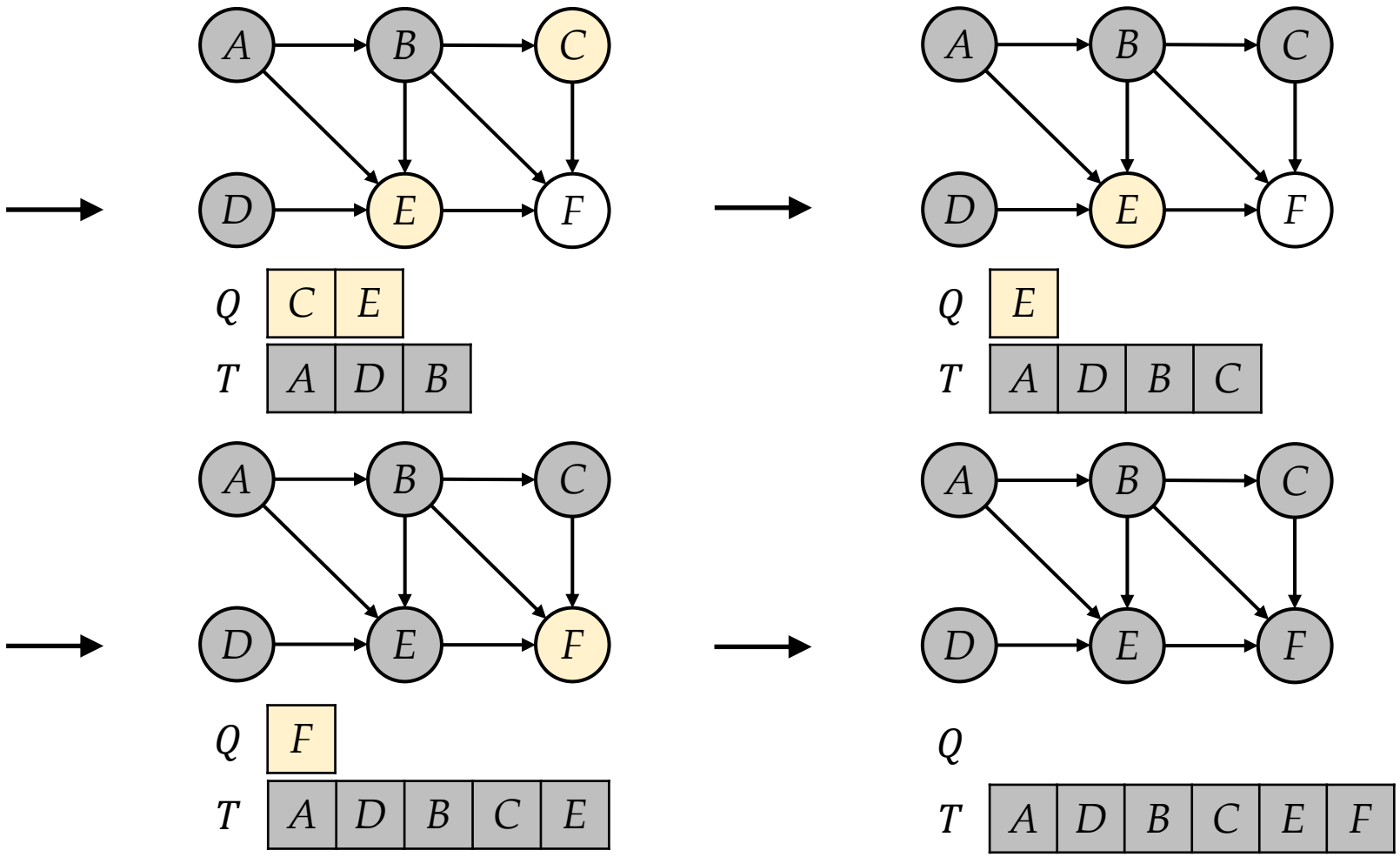
```
TopologicalSort(G)
  T ← empty list T
  Q ← empty queue Q
  for each v ∈ V
    if in-degree of v is 0
      Insert v into Q
  while Q is not empty
    u ← Pop element from Q
    Add u to T
    for each v ∈ V adjacent to u
      Decrease in-degree of v by 1
      if in-degree of v is 0
        Insert v into Q
  return T
```

- G is a DAG
- T stores the topological sort
- Either a queue or a stack
- Get the next vertex for the topological sort
- Update temporary in-degree and Q

Topological Sort



Topological Sort



Time Complexity

```
TopologicalSort(G)
  T ← empty list T
  Q ← empty queue Q
  for each v ∈ V
    if in-degree of v is 0
      Insert v into Q
  while Q is not empty
    u ← Pop element from Q
    Add u to T
    for each v ∈ V adjacent to u
      Decrease in-degree of v by 1
      if in-degree of v is 0
        Insert v into Q
  return T
```

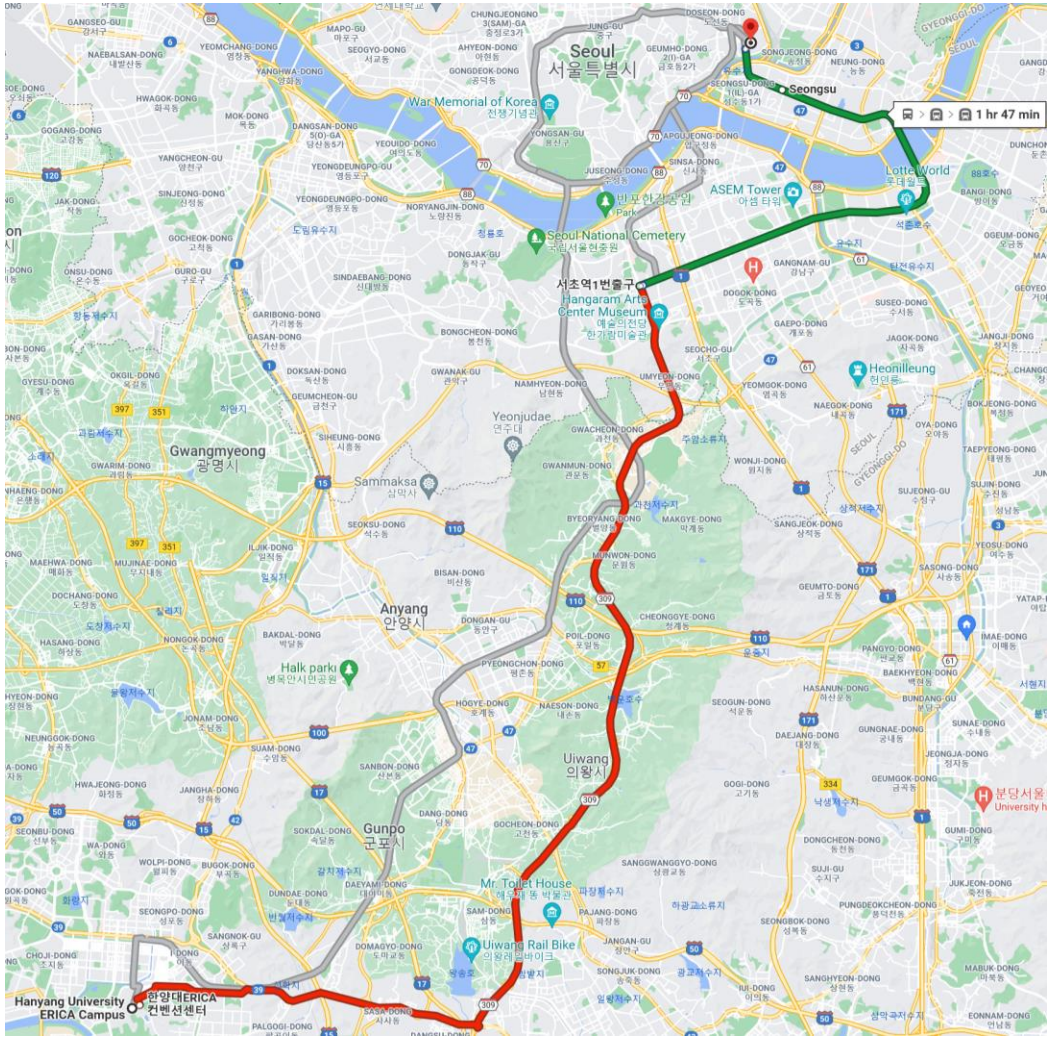
- $|V|$
- (Assume that we already store in-degree of v)
- $1 + indegree(v)$
- While loop iterates at most |V| times
- $\Sigma_{v \in V}\big(1 + indegree(v)\big) = |V| + |E|$
- Time complexity: $O(|V| + |V| + |E|) = O(|V| + |E|)$

The Shortest Path Problem

Find a shortest path between two vertices in a graph such that the sum of the weights of its constituent edges is minimized

The Shortest Path Problem

- Finding the fastest way to go from one place to another
- Finding the cheapest way to fly/send data from one city to another



The Shortest Path Problems

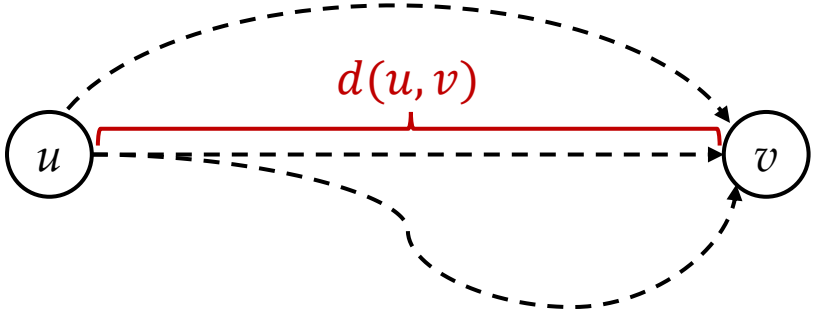
- Given a graph $G = (V, E)$
- **Single-Source Shortest Path Problem**
  - **Find a shortest path from a given source vertex $s \in V$ to each vertex $v \in V$.**
- Single-Destination Shortest Path Problem
  - Find a shortest path to a given destination vertex $t \in V$ from each vertex $v \in V$.
- Single-Pair Shortest Path Problem
  - Find a shortest path from u to v for given vertices $u \in V$ and $v \in V$.
- All-Pairs Shortest Path Problem
  - Find a shortest path from u to v for every pair of vertices $u \in V$ and $v \in V$.

Defining Shortest Path in a Weighted Graph

- **Weighted graph** is a graph that has a numeric label $w(u, v)$, the **weight of edge (u,v)**.
- The **length** of a path P is the sum of the weights of the edges in P.
  - Given a path $P = \big((v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)\big)$, the length of P is

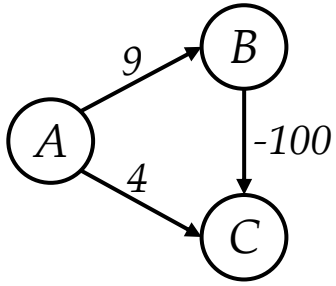
$$\Sigma_0^{k-1} w(v_i, v_{i+1})$$

- The **distance** $d(u, v)$ from vertex u to vertex v in G is the length of a minimum-length path (**shortest path**) from u to v.



Dijkstra's Algorithm

- Famous algorithm for the single-source shortest path problem
- Greedy method: always pick the next closest vertex to the source
- Use priority queue to store unvisited vertices by distance from s
- Do not work with negative weights on edges



Dijkstra's Algorithm

Dijkstra(G, s)

$D[v] \leftarrow \infty$ for each vertex $v \in V$

$D[s] \leftarrow 0$

Build a min priority queue Q with keys of $D[v]$ for all $v \in V$

while Q is not empty

$u \leftarrow$ Delete the minimum from Q

for each vertex v adjacent to u such that v is in Q

if $D[u] + w(u, v) < D[v]$

$D[v] \leftarrow D[u] + w(u, v)$

Update the key of vertex v in Q to $D[v]$

$P[v] \leftarrow u$

Relaxation

G: weighted graph with nonnegative edge weights

s: source vertex

Pull a new vertex u into the cloud

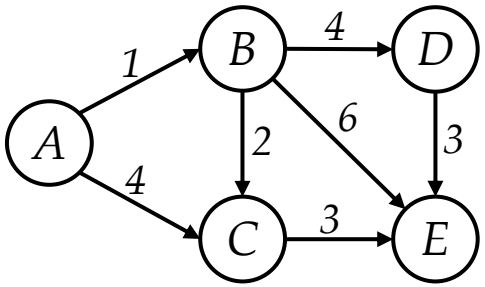
Perform relaxation on edge (u, v)

Output:

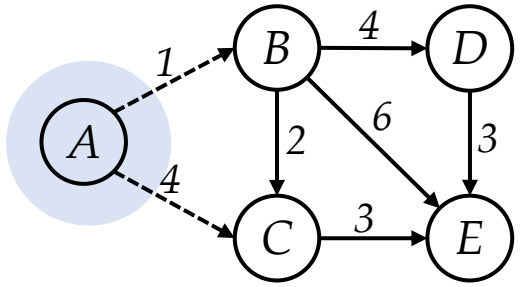
$D[v]$: the distance from s to v for every $v \in V$

$P[v]$: the predecessor of v in the shortest path, or the previous vertex that gave (updated) distance $D[v]$

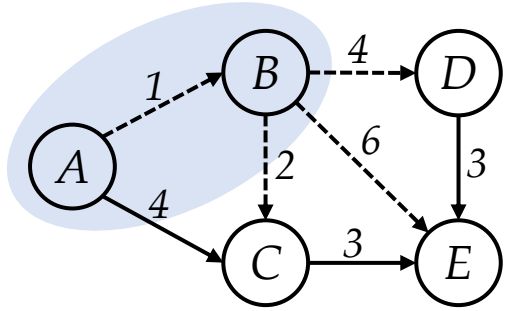
Dijkstra's Algorithm



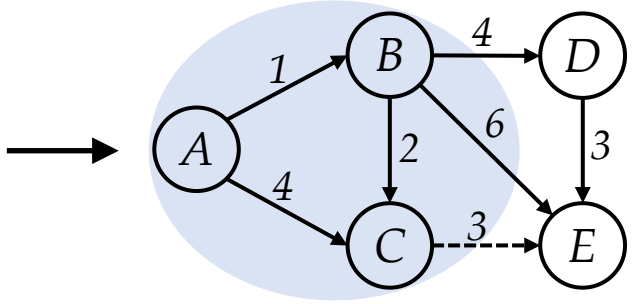
	A	B	C	D	E
D	**0**	∞	∞	∞	∞
P	-	-	-	-	-



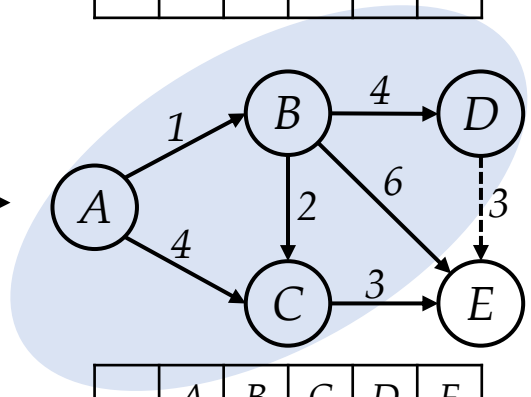
	A	B	C	D	E
D	0	**1**	**4**	∞	∞
P	-	**A**	**A**	-	-



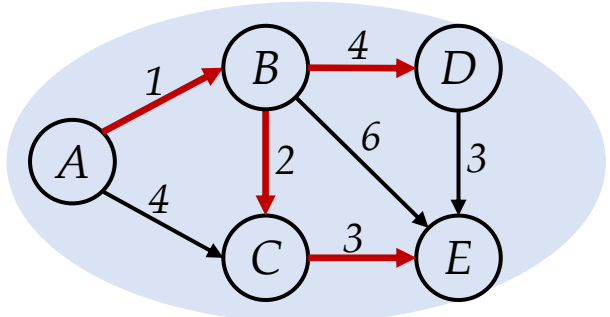
	A	B	C	D	E
D	0	1	**3**	**5**	**7**
P	-	A	**B**	**B**	**B**



	A	B	C	D	E
D	0	1	3	5	**6**
P	-	A	B	B	**C**



	A	B	C	D	E
D	0	1	3	5	6
P	-	A	B	B	C



	A	B	C	D	E
D	0	1	3	5	6
P	-	A	B	B	C

Time Complexity

Dijkstra(G, s)

$D[v] \leftarrow \infty$ for each vertex $v \in V$

$D[s] \leftarrow 0$

Build a min priority queue Q with keys of $D[v]$ for all $v \in V$

while Q is not empty

$u \leftarrow$ Delete min from Q

for each vertex v adjacent to u such that v is in Q

if $D[u] + w(u, v) < D[v]$

$D[v] \leftarrow D[u] + w(u, v)$

Update the key of vertex v in Q to $D[v]$

$P[v] \leftarrow u$

- $\Theta(|V|)$
- $\Theta(|V|)$
- For each vertex $v \in V$
- $\Theta(log|V|)$
- For each $outdegree(u)$
- $\Theta(log|V|)$
- Time complexity:
  $\Sigma_{v \in V}(log|V| + outdegree(u) \times log|V|)$
  $= |V|\, log|V| + log|V|\Sigma_{v \in V}\big(outdegree(u)\big)$
  $= (|V| + |E|)log|V| \in \boldsymbol{\Theta}((|\boldsymbol{V}| + |\boldsymbol{E}|)\boldsymbol{log}|\boldsymbol{V}|)$

Graph Algorithms

- Given a graph $G = (V, E)$

Operation	**Running Time**
BFS	$O(\|V\| + \|E\|)$
DFS	$O(\|V\| + \|E\|)$
Topological Sort	$O(\|V\| + \|E\|)$
Dijkstra's Algorithm	$O((\|V\| + \|E\|)log\|V\|)$